Assignment 3: Develop a distributed system, to find sum of N elements in an array by distributing N/n elements to n number of processors MPI or OpenMP. Demonstrate by displaying the intermediate sums calculated at different processors

Steps to Perform OpenMP Program on Ubuntu

Step 1: Open Ubuntu Terminal

Step 2: Install GCC Compiler

Update Ubuntu packages:

sudo apt update

Install GCC compiler:

sudo apt install gcc -y

Step 3: Create Project Folder

Create a new folder:

mkdir OPENMP

Move into the folder:

cd OPENMP

Step 4: Create Program File

Create a C program file:

nano sum.c

Step 5: Paste OpenMP Program

Paste the OpenMP program code into sum.c.

Save the file using:

Ctrl + O → Enter → Ctrl + X

Step 6: Compile the Program

Compile the program using OpenMP flag:

gcc -fopenmp sum.c -o sum

Step 7: Run the Program

Execute the program using:

./sum

\\\\ \\\\ \\\\ \\\\ \\\\ \\\\ \\\\ \\\\ \\\ \\\ \\\ \ \\\\\ \\\ \\ \

Viva Questions and Answers for OpenMP Assignment

1. What is a Distributed System?

A distributed system is a collection of multiple processors/computers working together to perform tasks.

---

2. What is OpenMP?

OpenMP is an API used for parallel programming using multiple threads/processors.

---

3. What is MPI?

MPI stands for:

Message Passing Interface

It is used for communication between multiple processors in distributed systems.

---

4. Why is OpenMP used in this assignment?

OpenMP is used to divide the array processing among multiple processors/threads to increase speed.

---

5. What is parallel programming?

Parallel programming means executing multiple tasks simultaneously using multiple processors or threads.

---

6. What is the purpose of this program?

To calculate the sum of array elements using multiple processors and display intermediate sums.

---

7. What does #pragma omp parallel for do?

It divides loop iterations among multiple threads automatically.

Example:

#pragma omp parallel for

---

8. What is reduction in OpenMP?

Reduction combines partial results from all threads into a single result.

Example:

reduction(+:sum)

---

9. What is omp_get_thread_num()?

It returns the ID number of the current thread/processor.

---

10. Which compiler is used for OpenMP?

GCC Compiler

---

11. Which compiler option enables OpenMP?

-fopenmp

---

12. Which command compiles the program?

gcc -fopenmp sum.c -o sum

---

13. Which command runs the program?

./sum

---

14. What is a thread?

A thread is the smallest unit of execution inside a process.

---

15. Difference between process and thread

Process	Thread
Heavyweight	Lightweight
Separate memory	Shared memory
Slower	Faster

---

16. What is shared memory?

Shared memory means all threads can access the same memory space.

---

17. Advantages of OpenMP

- Faster execution
- Easy parallel programming
- Efficient CPU utilization

---

18. Disadvantages of OpenMP

- Works mainly on shared memory systems
- Debugging can be difficult

---

19. Difference between MPI and OpenMP

MPI	OpenMP
Distributed memory	Shared memory
Multiple systems	Single system

MPI	OpenMP
Complex	Simple

---

20. What is the output of your program?

Processor 0 adding 10
Processor 1 adding 20
Processor 2 adding 30

Total Sum = 150

---

21. Why output order changes sometimes?

Because threads execute simultaneously in parallel.

---

22. Real-life applications of OpenMP

- Scientific Computing
- Machine Learning
- Weather Forecasting
- Data Processing

---

23. What is scalability?

Scalability means increasing performance by adding more processors.

---

24. What happens if OpenMP flag is not used?

Program compiles without parallel execution.

---

25. Which language is commonly used with OpenMP?

C / C++